

표지

팀명 : fiveguys / 전기공학과 / 발표일 26.06.04

역할분담 동일하게

목차 : 개요 / 부품 / 회로도 / 피드백 반영하며
성능 발전과정 나열 / 피드백 후 성능결과 차이
(지표) / 최종 로직 / 실제 구현 모델(좌표+모델
형상) + CAN 데이터 송수신의 결과
+ 특허&공모전 현황 + 향후계획 > Q&A

[프로젝트 개요]

PIEV 모델 기반 스마트 예측 조명 시스템

기존 문제점: 대시보드 조도 센서에만 의존하여, 터널 진입 시 3~5초의 점등 지연(암흑 상태) 발생

해결 방안: GPS 터널 위치 및 가속도 데이터 기반 0초 선제적 미등 제어 시스템 구축

💡 PIEV 핵심
P (인지 / Perception)

전방 터널 GPS 위치 인지 + 가속도 센서로 차량의 실제 주행 상태 감지

I (판단 / Intellection)

ESP32 제어기가 데이터를 연산하여 "터널 진입 전 미등 점등 필수"로 정의

E (감정 / Emotion)

터널 진입 순간 암흑으로 인한 운전자의 시각적 불안감(공포) 완벽 해소

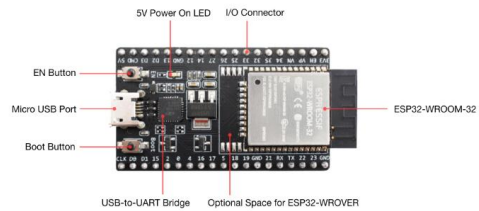
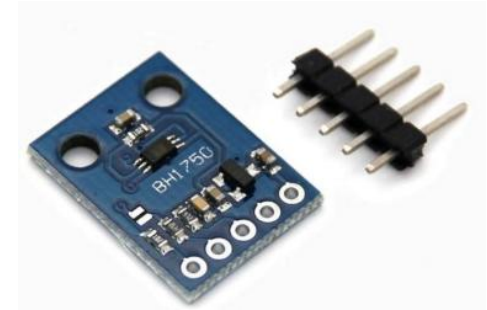
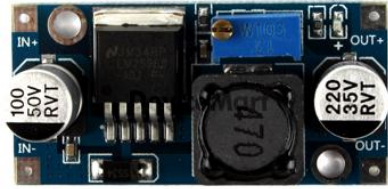
V (실행 / Volition)

터널 진입 0초 만에 MOSFET 하드웨어 스위칭으로 미등 회로 즉시 구동

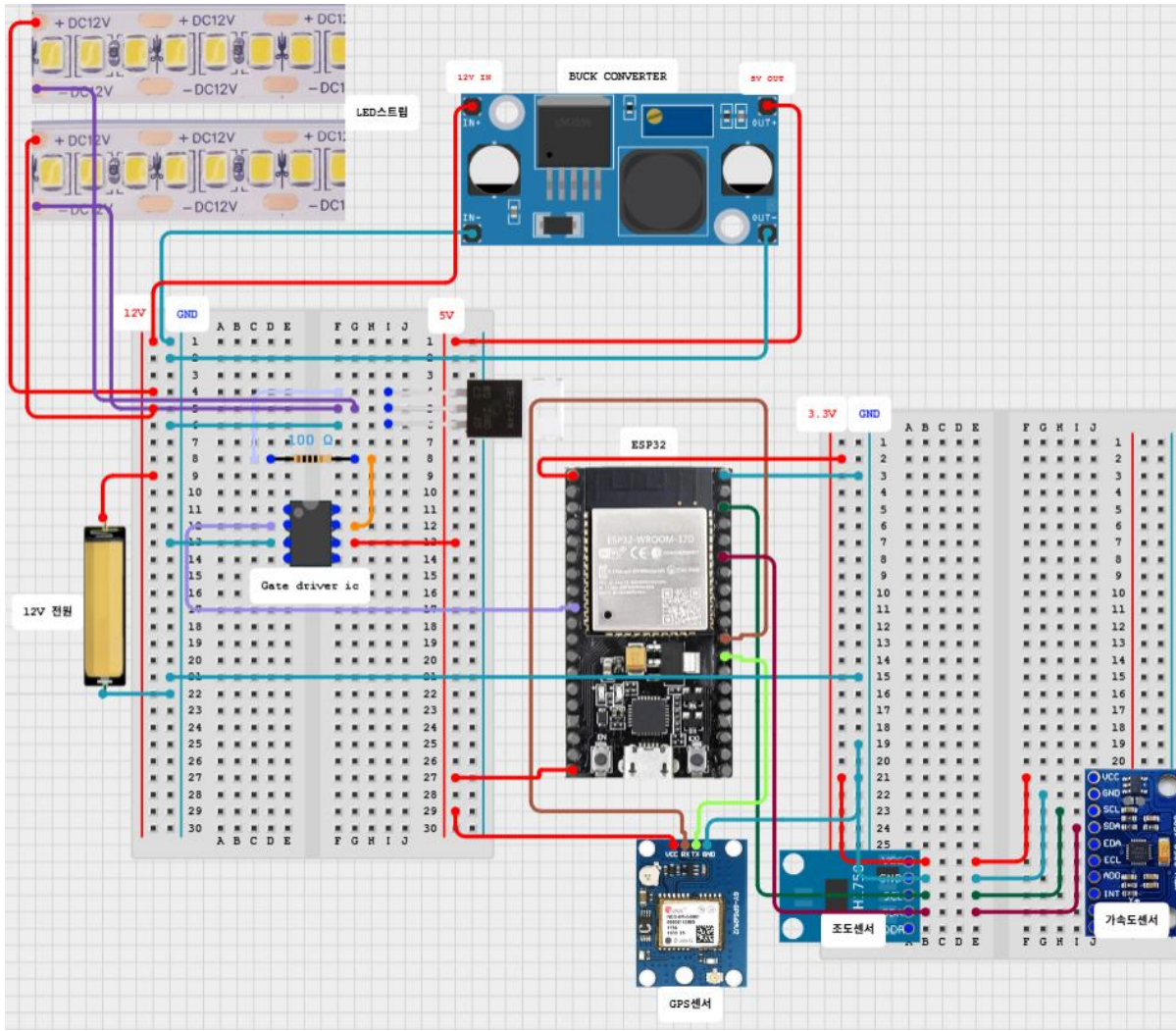
🎯 한 줄 요약

"오토라이트의 3~5초 딜레이를 GPS 예측 기술로 극복하여, 터널 진입 순간의 안전 공백을 0초로 만드는 기술"

주요소자 buck converter / 조도센서(BH1750) /
자이로가속도센서 / GPS 모듈 (NEO-M8N) / ESP32 devkit v4(mcu) / 게이트드라이버IC (TC4427)



회로도

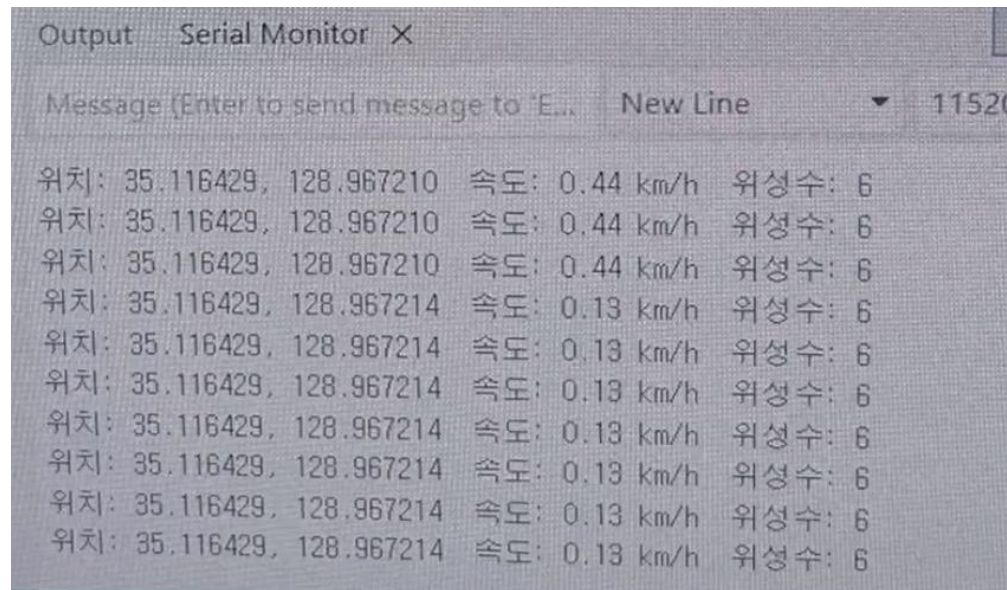


발전과정

1. GPS만 달았을 때 -> 위성수 6개 기준으로 정지시에도 GPS의 지점이 고정되지 않아서 계속 이동하는 문제가 존재하였음. 이때의 오차범위는 그린필드를 기준으로 정지시 5m~크게는 이동시 20m까지도 확인가능하였음.

결과적으로 위성의 수신기 모듈이 가능한 최대갯수인 12개에서 가장 안정적으로 동작하며 이를 위해서 실내인 학교 복도는 제한이 있다고 판단. + 자이로/가속도센서의 결합으로 센서 퓨전을 통해 종합적 오차를 줄이기로 결심 / 움직일때도 좌표의 떨림이 심하였음

(사진 : 정지시 GPS모듈만 사용했을때 데이터)




```

43 Serial.print("- Y: ")
44 Serial.print(y);
45 Serial.println("");
46 delay(200);
47 }

```

Output Serial Monitor X

Message (Enter to send message to 'ESP32 Dev Module' on 'COM3')

```

Acc X:0.37 Y:0.18 Z:11.34
Acc X:0.37 Y:0.19 Z:11.35
Acc X:0.39 Y:0.19 Z:11.37
Acc X:0.36 Y:0.12 Z:11.36
Acc X:0.38 Y:0.26 Z:11.35
Acc X:0.37 Y:0.14 Z:11.38
Acc X:0.39 Y:0.19 Z:11.37
Acc X:0.36 Y:0.18 Z:11.37
Acc X:0.35 Y:0.17 Z:11.34
Acc X:0.34 Y:0.18 Z:11.35
Acc X:0.37 Y:0.19 Z:11.35
Acc X:0.41 Y:0.17 Z:11.33
Acc X:0.37 Y:0.21 Z:11.35
Acc X:0.36 Y:0.20 Z:11.37
Acc X:0.37 Y:0.17 Z:11.35
Acc X:0.36 Y:0.18 Z:11.37
Acc X:0.36 Y:0.19 Z:11.37
Acc X:0.33 Y:0.19 Z:11.41
Acc X:0.39 Y:0.20 Z:11.38

```

```

17 Wire.beginTransaction(0x68);
18 Wire.write(0x6B); Wire.write(0x00);
19 Wire.endTransmission();
20 }
21 }
22
23 void loop() {
24   sensors_event a, g, temp;

```

Output Serial Monitor X

Message (Enter to send message to 'ESP32 Dev Module' on 'COM3')

```

ACC: 0.01, 0.04, 9.79 | GYRO: 0.00, 0.05, -0.01
ACC: 0.04, 0.04, 9.79 | GYRO: 0.00, 0.05, -0.01
ACC: 0.03, 0.09, 9.79 | GYRO: 0.00, 0.05, -0.01
ACC: 0.01, 0.04, 9.80 | GYRO: 0.00, 0.04, -0.01
ACC: 0.04, 0.06, 9.80 | GYRO: 0.00, 0.04, -0.01
ACC: 0.03, 0.06, 9.82 | GYRO: 0.00, 0.05, -0.01
ACC: 0.02, 0.07, 9.79 | GYRO: 0.00, 0.05, -0.01
ACC: 0.04, 0.04, 9.81 | GYRO: 0.00, 0.05, -0.01
ACC: 0.05, 0.08, 9.78 | GYRO: 0.00, 0.04, -0.01
ACC: 0.03, 0.04, 9.78 | GYRO: 0.00, 0.05, -0.01
ACC: 0.02, 0.05, 9.80 | GYRO: 0.00, 0.04, -0.01
ACC: 0.06, 0.10, 9.79 | GYRO: 0.00, 0.04, -0.01
ACC: 0.01, 0.04, 9.81 | GYRO: 0.00, 0.05, -0.01
ACC: 0.04, 0.03, 9.81 | GYRO: 0.00, 0.04, -0.01
ACC: 0.05, 0.07, 9.83 | GYRO: 0.00, 0.05, -0.01
ACC: 0.04, 0.05, 9.80 | GYRO: 0.00, 0.05, -0.01
ACC: 0.03, 0.05, 9.78 | GYRO: 0.00, 0.04, -0.01
ACC: 0.05, 0.06, 9.80 | GYRO: 0.00, 0.04, -0.01
ACC: 0.02, 0.03, 9.79 | GYRO: 0.01, 0.04, -0.01

```

2. GPS + 자이로/가속도 센서

1) 자이로·가속도 센서를 장착하고 GPS와 데이터 결합만 해둔 상태 : 센서자체의 오차가 존재 -> 상시 움직이는 중으로 오인 -> 정지시에도 GPS의 드리프트 존재 -> 가속도가 변하는 분산 자체는 미미함.-> 정지시 데이터점 수집 후 평균을 내어 코드에 소자의 보정치(오프셋)대입

2) 하드웨어 오프셋 적용 및 주행 상태 필터링

해당 수식 이용해 정렬

$$\sqrt{X^2 + Y^2 + Z^2}$$

자이로는 차가 언덕이나 중력의 방향이 바뀔 경우 회전 각속도를 감지해 9.8에서 벗어나면 그에 따라 보정이 되도록 필터링하여 정지시를 판별하였음

주행필터링은 이 제공값의 제공근이 0.6보다 클때 이동으로 인식하도록 하여 주행필터링완료

결국 GPS오차가 최대 15m에서 평균적으로 1~1.5m가 나오는 걸 확인하였음.

Acc x,y,z 만 나와잇는사진 - 보정전 가속도센서 데이터
acc/gyro - 보정후

조도 센서

- 미등제어 로직 활성을 위한 밤낮구분을 위해 기준을 세울 필요 존재

-> 기준값을 하나로 선정시 하나의 부근에서 진동시 시스템의 채터링 발생(예시사진 있으면 좋을듯)

-> 기준값을 두 개로 나눔 ->

차량 내 켜팅 보정 및 실제 측정값들을 기준으로 낮 : 3500이상 2초 유지시 낮으로 판별

밤 : 500이하 2초 이상 유지시 밤으로 판별

Bh1750 측정 지표(sat : 65,535 (16bit)) 밤일때 로직 활성화가 되지 않으면 되므로 max값은 ㄱ

태양빛(창문x)

-밝은 낮 : 약50000

-그늘: 약3000

-완전 차단:약100~200

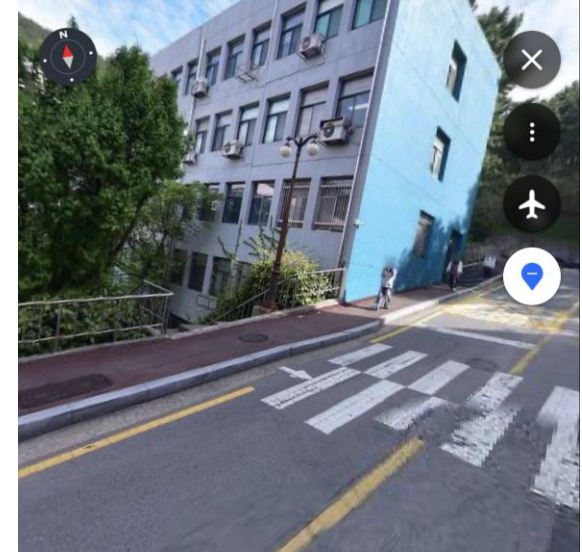
실내(태양빛거의x)

-형광등on:약600

-약간 차단:약200

-완전 차단:약10~50

앞의 내용 기반 센서보정 전후 결과 지표 및 자료 증빙
지표만들어주세요 좀 그럴싸하게



```

#include <TinyGPS++.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
#include <BH1750.h>

TinyGPSPlus gps;
Adafruit_MPU6050 mpu;
HardwareSerial gpsSerial(2);
BH1750 lightMeter;

// =====
// 🚀 [Secter 1] 변수 선언 및 타겟 설정
// =====

const double TARGET_LAT = 35.105925;
const double TARGET_LNG = 129.007105;

const float BIAS_X = 2.47;
const float BIAS_Y = -1.53;
const float BIAS_Z = 1.19;

const int ledPin = 25;
const int ledChannel = 0;
const int ledFreq = 5000;
const int ledRes = 8;

const float NIGHT_LIMIT = 800.0;

bool mpuReady = false;

const int AVG_SIZE = 10;
double latBuffer[AVG_SIZE], lngBuffer[AVG_SIZE];
int bufIdx = 0;
double lastValidLat = 0, lastValidLng = 0;
bool firstFix = false;

```

```

//
=====
// 하드웨어 초기화 및 세팅//
=====

void setup() {
  Serial.begin(115200);
  delay(1500);

  #if ESP_ARDUINO_VERSION_MAJOR < 3
    ledcSetup(ledChannel, ledFreq, ledRes);
    ledcAttachPin(ledPin, ledChannel);
  #else
    ledcAttach(ledPin, ledFreq, ledRes);
  #endif

  gpsSerial.begin(9600, SERIAL_8N1, 16, 17);
  Wire.begin(21, 22);
  lightMeter.begin();

  if (mpu.begin()) {
    mpuReady = true;
    Serial.println("✅ 가속도 센서(MPU6050) 필터링 준비 완료");
  } else {
    Serial.println("⚠️ 가속도 센서 연결 실패! 필터링 없이 진행합니다.");
  }
}

void loop() {
  unsigned long currentTime = millis();

  while (gpsSerial.available() > 0) {
    gps.encode(gpsSerial.read());
  }
}

```

[Secter 1] 환경 설정 구역: 시연 장소인 부산 대
티터널의 정밀 좌표와 MPU6050 센서의 정지 상
태 오차를 보정하기 위한 핵심 상수(BIAS)들을
선언하는 곳입니다.

```

//
=====
// 🛠️ [Secter 2] 1차 물리 필터:
//   가속도 기반 주행 감지
//
=====
bool isMoving = true;
if (mpuReady) {
    sensors_event_t a, g, temp;
    mpu.getEvent(&a, &g, &temp);
    float adjX = a.acceleration.x -
        BIAS_X;
    float adjY = a.acceleration.y -
        BIAS_Y;
    float adjZ = a.acceleration.z -
        BIAS_Z;
    float motionMag =
    sqrt(adjX*adjX + adjY*adjY + (adjZ
    - 9.8)*(adjZ - 9.8));
    isMoving = (motionMag > 0.6);
}

```

```

// =====
// 🛠️ [Secter 2] 1차 물리 필터: 가속도 기반 주행 감지
// =====
bool isMoving = true;
if (mpuReady) {
    sensors_event_t a, g, temp;
    mpu.getEvent(&a, &g, &temp);
    float adjX = a.acceleration.x - BIAS_X;
    float adjY = a.acceleration.y - BIAS_Y;
    float adjZ = a.acceleration.z - BIAS_Z;
    float motionMag = sqrt(adjX*adjX + adjY*adjY + (adjZ - 9.8)*(adjZ -
    9.8));
    isMoving = (motionMag > 0.6);
}
// =====
// 📍 [Secter 3] 2차 위치 융합: GPS 이동평균 및 거리 연산
// =====
double distanceToTarget = 9999.0;
if (gps.location.isValid() && gps.satellites.value() >= 4) {
    double rawLat = gps.location.lat();
    double rawLng = gps.location.lng();

    if (!firstFix) {
        for(int i=0; i<AVG_SIZE; i++) { latBuffer[i] = rawLat; lngBuffer[i] =
        rawLng; }
        firstFix = true;
        lastValidLat = rawLat; lastValidLng = rawLng;
    }
    else if (isMoving) {
        latBuffer[bufIdx] = rawLat; lngBuffer[bufIdx] = rawLng;
        bufIdx = (bufIdx + 1) % AVG_SIZE;
        double sumLat = 0, sumLng = 0;
        for(int i=0; i<AVG_SIZE; i++) { sumLat += latBuffer[i]; sumLng +=
        lngBuffer[i]; }
        lastValidLat = sumLat / AVG_SIZE; lastValidLng = sumLng / AVG_SIZE;
    }
}

if (firstFix) {
    distanceToTarget = TinyGPSPlus::distanceBetween(lastValidLat,
    lastValidLng, TARGET_LAT, TARGET_LNG);
}

```

[Secter 2] 가속도 필터링 구역: 오프셋이 보정된 3축 가속도 값을 기하학적으로 합성한 후, 중력을 상쇄시켜 오직 차의 순수 움직임 크기(motionMag)만 추출합니다. 이를 통해 차가 정지해 있는지 주행 중인지 칼같이 분별합니다.

[Secter 3] GPS 융합 구역 (Sensor Fusion): 가속도 필터가 주행 중 (isMoving)이라고 판단할 때만 GPS 좌표 버퍼를 갱신하고 10차 이동평균 필터를 거치게 만듭니다. 신호 대기 중 GPS 혼자 사방으로 튕겨서 오작동하는 현상을 완벽하게 봉쇄하고 터널 입구와의 정확한 남은 거리를 계산합니다.

```

//
=====
// 📍 [Secter 4] 조건 분리형 스마트
// 미등 제어 (조도 센서 측정 포함)
//
=====

float lux =
lightMeter.readLightLevel();
int brightness = 0;

if (lux <= NIGHT_LIMIT && lux >= 0) {
    brightness = 255;
}
else {
    if (!firstFix || distanceToTarget >=
300.0) {
        brightness = 0;
    }
    else if (distanceToTarget < 300.0 &&
distanceToTarget > 45.0) {
        brightness = (int)((300.0 -
distanceToTarget) * (255.0 / (300.0 -
45.0)));
    }
    else if (distanceToTarget <= 45.0) {
        brightness = 255;
    }
}

```

```

// =====//
// 📍 [Secter 5-B] 하드웨어 드라이빙 및 모니터링 출
// 력
// =====

brightness = constrain(brightness, 0, 255);

#if ESP_ARDUINO_VERSION_MAJOR < 3
    ledcWrite(ledChannel, brightness);
#else
    ledcWrite(ledPin, brightness);
#endif

static unsigned long lastPrint = 0;
if (currentTime - lastPrint > 1000) {
    if (lux <= NIGHT_LIMIT && lux >= 0) {
        Serial.print("[ 🌙 야간 모드 - 상시 점등]");
    } else {
        Serial.print("[ 🌞 주간 모드 - 거리 제어]");
    }

    Serial.print(" 조도: "); Serial.print(lux, 0);
    if (firstFix) {
        Serial.print("lx | 거리: ");
        Serial.print(distanceToTarget, 1); Serial.print("m");
    } else {
        Serial.print("lx | 거리: 위성 대기 중...");
    }
    Serial.print(" | 미등 밝기: ");
    Serial.print((int)((brightness / 255.0) * 100));
    Serial.print("%");
    Serial.print(" (PWM: "); Serial.print(brightness);
    Serial.println(")");
    lastPrint = currentTime;
}
}

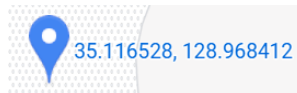
```

[Secter 4] 지능형 제어 구역: 밤에
는 조도센서 기반 상시 점등(100%),
낮에는 터널 전방 300m부터 거리
가 가까워질수록 밝기가 부드럽게
선형 증가(0% → 100%)하여 터널
진입 전 완전히 시야를 확보하는
상황 인지형 알고리즘 구조입니다.

[Secter 5] 하드웨어 및 모니터링 구
역: 계산된 밝기를 5kHz 고주파
PWM 신호로 변환하여 모스펫 드
라이버 회로를 실시간 스위칭하고,
시스템 지연을 유발하는 delay() 없
이 millis() 타이머를 활용해 시연
데이터를 1초 간격으로 모니터에
뿌려주는 최종 출력부입니다.

대티터널에서 시물시 진행 영상 + 하드웨어 실물

구현 설명(좌표 / 영상)
가상의 터널 시작점 좌표 -> 이를 기반으로 우리는 실험을 할것이다.



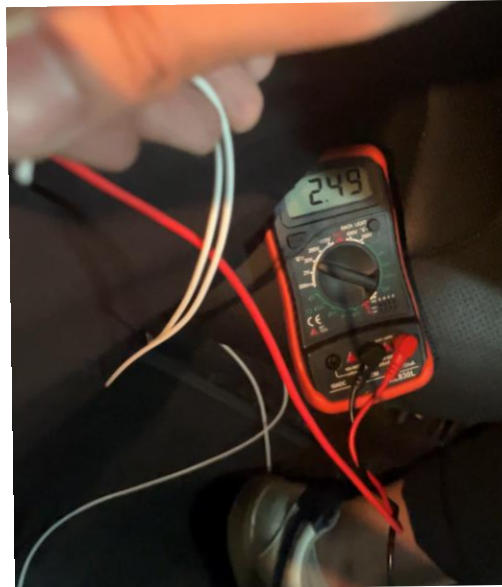
CAN 진행결과

B-CAN (Body CAN): 차량의 파워트레인(엔진, 변속기)이 아닌, 바디 전장 시스템(도어록, 와이퍼, 창문, 그리고 '미등/헤드라이트')을 제어하기 위한 통신 규격

핵심 원리: 차량 내부는 엔진, 발전기 등 엄청난 전기적 노이즈가 발생하는 공간입니다. 이를 극복하기 위해 B-CAN은 선 한 개가 아니라 두 개의 선(CAN_High, CAN_Low)을 꼬아서 신호를 보내는 차동 전압(Differential Voltage) 방식을 사용

대기 (1) : 2.5/2.5

구동 (0) : 3.5/1.5



CAN 진행결과

우리가 구현하려던 차량 : 아반떼 CN7 22년식

ESP32 제어기에서 연산한 터널 거리 데이터를 기반으로, 이 B-CAN 라인에 "미등을 켜라"는 디지털 패킷(ID 및 데이터)을 주입하여 조명을 제어하는 것을 최종 목표

1. OBD 데이터 수신 -> 차량내 IBU 방화벽으로 인해 특정 데이터 이외의 값 수신 불가
원인 분석 (보안 게이트웨이의 장벽):

최신 차량(아반떼 CN7)은 해킹 방지 및 차량 사이버 보안을 위해 보안 게이트웨이(SGW, Security Gateway)라는 강력한 방화벽이 OBD2 단자 바로 뒷단에 배치되어 있습니다

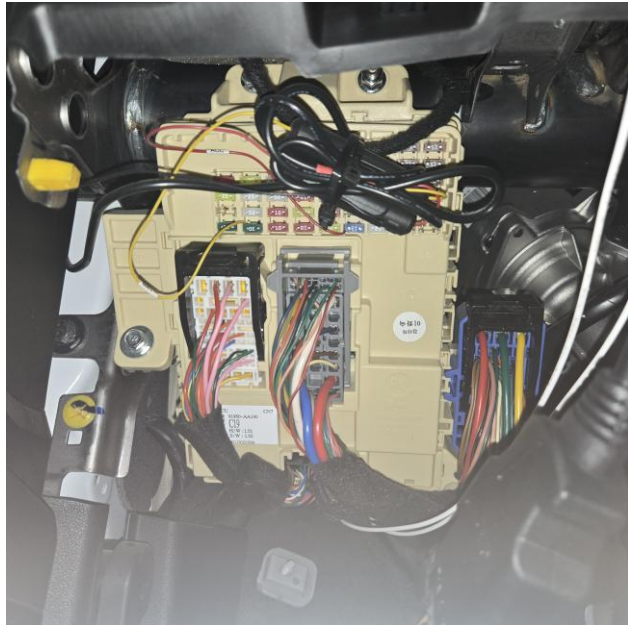
현재 차량의 게이트웨이는 허가되지 않은 DIY 모듈이나 진단 장비가 접속하면 신호를 완전히 차단(Filtering)하거나, 특정 데이터 요청 쿼리(Request)를 엄격한 보안 프로토콜에 맞춰 날리지 않으면 B-CAN 데이터 자체를 외부로 내보내지 않는 '폐쇄형 구조'를 취함.

사진 : oxAA들 나와잇는거 + OBD2단자사진

CAN 진행결과

2. T-tap 이용 수신 -> 성공

1) 데이터 수신 확인(총 33개의 데이터)



Output Serial Monitor X

Message (Enter to send message to 'ESP32 Dev Module' on 'COM3')

```
ID: 0x101 | 데이터: 0x10 0x01 0x00 0x00 0x00 0x00 0x00 0x00
ID: 0x112 | 데이터: 0x00 0x88 0x08 0x5F 0x04 0x80 0x00 0x00
ID: 0x189 | 데이터: 0x00 0x00 0x00 0x04 0x00 0x2B 0x00 0x40
ID: 0x102 | 데이터: 0x00 0x00 0xA2 0x00 0x00 0x00 0x00 0x00
ID: 0x113 | 데이터: 0x10 0x00 0x00 0x00 0x00 0x00 0x00 0x00
ID: 0x103 | 데이터: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
ID: 0x510 | 데이터: 0x02 0xD0 0x00 0x00 0x00 0x00 0x00 0x00
ID: 0x189 | 데이터: 0x00 0x00 0x00 0x04 0x00 0x28 0x00 0x40
ID: 0x400 | 데이터: 0x16 0x02 0x00 0x00 0x00 0x00 0xFF 0xFF
ID: 0x104 | 데이터: 0x00 0x40 0x00 0x02 0x00 0x34 0x00 0x00
ID: 0x105 | 데이터: 0x00 0x00 0x00 0x00 0x08 0x00 0x00 0x00
ID: 0x189 | 데이터: 0x00 0x00 0x00 0x04 0x00 0x28 0x00 0x40
ID: 0x18F | 데이터: 0x00 0x00 0x40 0x00 0x00 0x00 0x00 0x00
ID: 0x168 | 데이터: 0x08 0x00 0x04 0x01 0x21 0x30 0x00 0x00
ID: 0x1F7 | 데이터: 0x00 0x00 0x00 0x40 0x00 0x00 0x00 0x00
ID: 0x106 | 데이터: 0x00 0x00 0x00 0x00 0x00 0x3E 0x3C 0x4C
ID: 0x1FA | 데이터: 0x15 0x80 0x00 0x00 0x00 0x00 0x00 0x00
ID: 0x169 | 데이터: 0x00 0x20 0x00 0x00 0x08 0x40 0x00 0x00
ID: 0x16A | 데이터: 0x00 0x00 0x00 0x00 0x02 0x00 0x00 0x00
ID: 0x107 | 데이터: 0x00 0x00 0x00 0x20 0x00 0x00 0x00 0x00
ID: 0x18A | 데이터: 0x00 0x00 0x00 0x00 0x00 0x00 0x30 0x00
ID: 0x1BD | 데이터: 0x44 0x00 0x00 0x00 0x00 0x00 0x00 0x00
ID: 0x18B | 데이터: 0x14 0xFF 0xF8 0x00 0xEF 0x00 0x0C 0x00
ID: 0x18C | 데이터: 0x82 0x00 0x00 0x00 0x00 0x00 0x00 0x00
ID: 0x416 | 데이터: 0x00 0x02 0x00 0x00 0x00 0x00 0xFF 0xFF
ID: 0x18D | 데이터: 0x10 0xE0 0x5E 0x5E 0x5D 0x80 0x00 0x08
ID: 0x189 | 데이터: 0x00 0x00 0x00 0x04 0x00 0x29 0x00 0x40
ID: 0x110 | 데이터: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
ID: 0x18E | 데이터: 0xFF 0xFF 0xFF 0xFF 0x00 0x00 0x00 0x00
ID: 0x100 | 데이터: 0x84 0x00 0x00 0xD8 0x00 0x04 0x80 0x00
ID: 0x111 | 데이터: 0x80 0x00 0x00 0x00 0x00 0x00 0x00 0x00
ID: 0x189 | 데이터: 0x00 0x00 0x00 0x04 0x00 0x2A 0x00 0x40
ID: 0x101 | 데이터: 0x10 0x01 0x00 0x00 0x00 0x00 0x00 0x00
ID: 0x112 | 데이터: 0x00 0x88 0x08 0x5F 0x04 0x80 0x00 0x00
ID: 0x102 | 데이터: 0x00 0x00 0xA2 0x00 0x00 0x00 0x00 0x00
```

CAN 진행결과

2) 변화하는 데이터만 빼는 로직 이용

(💡 변화 감지! ID: 0x101 | 데이터: 0x10 0x01 0x00 0x00 0x00 0x00 0x00 0x00
💡 변화 감지! ID: 0x107 | 데이터: 0x00 0x00 0x00 0x20 0x00 0x00 0x00 0x00
💡 변화 감지! ID: 0x510 | 데이터: 0x02 0xD0 0x00 0x00 0x00 0x00 0x00 0x00
💡 변화 감지! ID: 0x102 | 데이터: 0x00 0x00 0xA2 0x00 0x00 0x00 0x00 0x00
💡 변화 감지! ID: 0x168 | 데이터: 0x08 0x00 0x04 0x01 0x21 0x30 0x00 0x00
💡 변화 감지! ID: 0x189 | 데이터: 0x00 0x00 0x00 0x04 0x00 0x28 0x00 0x40)

3) 환경 통제시 변화하는 데이터 소거(미등 상태 변화 x)

189 510 112 101 102 107

CAN 진행결과

4)168 pick과 그 이유

라이트 레버를 돌리는 순간, 데이터가 0x01에서 0x89로 변환
이진수로 변환시

미등 OFF (0x01) → 0000 0001

미등 ON (0x89) → 1000 1001

미등제어 비트는 첫번째라는 것까지 확인

3. 미등 점등 명령 상태의 데이터 송신시 미등이 제어되지않았음.

원인분석 :정상 순정 상태: 차량의 정품 바디제어모듈(BCM)이 주간 환경에서 10ms~20ms의 미등 off 데이터를 보내는중

우리 모듈의 주입: 이때 우리 ESP32 모듈이 "미등 켜짐(ON)" 패킷을 중간에 5ms주기로 억지로 끼워 넣었습니다.

결과:

1. '비트(Bit)' 단위의 물리적 충돌과 상쇄

우세 비트(Dominant, 0): 전압 차이 2V를 만듦 (힘이 강함)

열성 비트(Recessive, 1): 전압 차이 0V, 대기 상태 (힘이 약함)

2.CU의 즉각적인 '에러 처리' 및 버스 보호

ICU 컴퓨터는 B-CAN 버스의 절대적인 주도권을 쥔 '마스터'입니다. ICU는 자기가 분명히 미등 OFF 패킷(0)을 쏘았는데, 어떤 정체 모를 모듈(우리 ESP32)이 5ms 주기로 미등 ON 패킷(1)을 쏘며 데이터를 오염시키려고 하면 이를 '통신 에러(Bit Error)'로 감지 -> 순정상상태 유지

가능성

1. 법규 허용범위 내 자동 제어 합법

핵심요건 : 켜져있어야할상황에서 꺼지면 안됨
규정된 최소 광도 이하로 떨어지면 안됨.
운전자가 임의로 조작할수없어야함.

2. 미등 밝기 제어 전달 가능 + 현재 미등 밝기 제어의 표준이 PWM임.

3. '**하드웨어가 없는 경우'**란, 단순히 IPS 소자가 박혀있는가 여부를 넘어 '**전장 회로의 물리적 연결성**'을 의미합니다.

즉, BCM 메인 컴퓨터에서 IPS 칩까지 고속 주파수를 제어할 수 있는 PWM 전용 하드웨어 핀과 회로 라인이 기판에 동선으로 깔려 있지 않거나, 탑재된 IPS 소자와 램프 드라이버가 고속 스위칭(디밍)을 지원하지 않는 물리적 사양일 때를 말합니다.

이런 하드웨어적 기반이 아예 생략된 차량은 소프트웨어(펌웨어) 업데이트만으로는 절대로 미등 밝기 제어를 구현할 수 없습니다."

질문:

1. 현재 차량에 DRL(주간주행등)이 **ips 소자**를 사용 하여 밝기를 제어하는걸로아는데 미등도 제어가능한가요?

2. 구현 및 테스트 하기위해서 실제 차량에 **b캔 통신을 사용해서 미등 밝기 제어**가 가능한가요?
b캔 말고도 다른 통신방법으로 제어가 가능한가요?
(mcp2515 라는 트랜시버 및 캔 컨트롤러로 b캔 선에 병렬연결후 제어 할려고합니다)

3. 위에 두개가 가능할때 차량 제조사쪽에서는 **자체 펌웨어 업데이트**만으로도 ips 소자가 들어가있는 차들은 밝기 제어가 가능한가요?

구분	내용	적용 용이성
법적 규제	국토부 지능형 교통시스템(ITS) 및 자동차 안전 기준 적용 가능	높음 (안전 공백 0초 달성)
신규/최신 차량	BCM 펌웨어 업데이트만으로 하드웨어 추가 없이 즉시 적용	최상 (제조사 비용 제로)
기존 구형 차량	본 프로젝트에서 제안한 하드웨어 (MOSFET 소자) 모듈 장착 구동	보통 (애프터마켓 시장 형성 가능)

특허 / 공모전 진행과정

1. 공모전

교통·모빌리티·항공 분야 (자율주행, 도심항공교통(UAM) 등 주행/ITS 연관 분야 포함)

1차 분류 및 의견 정리 진행중

6,7월 중으로 선정예정

평가 기준 (100점 만점)

국민편익 및 파급효과 (40점): 제안을 통해 실제 생활 속 규제가 얼마나 해소되고 효과가 큰지 평가

창의성 (30점): 아이디어가 참신하고 독창적인지 평가

실현가능성 (30점): 실제 법규 개정이나 하드웨어/소프트웨어 기술 개발을 통해 구현할 수 있는지 평가

아 예 선생님 그 터널 사고 방지를
위한 미딩 제어 시스템 조금
맞을까요?

아 네 맞습니다.

아 예 잘 접수됐고요.

그 저희가 건수가 워낙 많이
들어와서 지금 400여 건이 넘게
들어와서 저희가 정리를 하고 있고
그 우리 부서 의견을 좀 정리를
된 다음에 지금 민간 위원으로
구성된 위원회에 거쳐서 선정을
할 계획이고 그게 67월 정도 해야
가능할 것 같습니다.

특허 / 공모전 진행과정

2. 특허
가출원해놓았음.

26. 4. 28. 오후 3:15

특허원III

특허고객번호 통지서

접 수 번 호 : 4-1-2026-5129843-85
접수담당자 :
서류제출자 : 이승주
등 록 일 자 : 2026.04.06
특허고객번호 : 4-2026-026500-3

기 초 정 보	성명	한글성명			이승주		
		영문성명			LEE,SEUNG JU		
		주민등록번호		000326-3*****	출원인구분	국내자연인	
		사업자등록번호		없음			
		전화번호		010-2261-****	핸드폰번호	없음	
	Email주소		lo*****@naver.com				
주 소 정 보	거 소	거주국	KR 대한민국				
		우편번호	50648	시도·국적	10 경상남도		
		신청주소(한글)	없음				
		변경주소(한글)	경상남도 양산시				
		신청주소(영문)	없음				
	송 달	송달장소우편번호	없음				
		신청주소(한글)	없음				
	변경주소(한글)	없음					
인		감			서		명

특허법 제28조의 2·실용신안법 제3조·디자인보호법 제29·상표법 제29조
규정에 따라 위와 같이 출원인코드를 통지합니다.

지식재산처장

관인생략

about:blank

발송번호: 1-5-2026-007767946
발송일자: 2026.04.11.

담당	사무관	과장	국장	차장	차장	결재
	전결					

지식재산처
청구범위 제출유예 안내서

출원인성명 이승주(4-2026-026500-3)
주소 경상남도 양산시 물금읍 황산로 753, 104동 105호(양산더 포레스트 엠)

대리인성명
주소

출원번호 10-2026-0065642 (출원일 : 2026.04.10)

발명의명칭 터널사고 방지를 위한 미등제어 시스템
제출기한 2027.06.10

- 위 출원은 특허법 시행규칙 제21조 제5항에 따른 임시 명세서를 첨부하여 청구범위 제출을 유예(출원 당시에 청구범위를 기재하지 아니한 명세서를 출원서에 첨부)하였으므로 위의 제출기한까지 청구범위를 제출하여야 합니다.
(관계 법령: 「특허법」 제42조의2제2항)
- 출원한 경우 청구범위 제출서식은 「특허법 시행규칙」 별지 제9호 서식의 「보정서」[보정구분:임시 명세서 보정]입니다.
- 보정서에 별지 제15호 서식·별지 제16호 서식 및 별지 제17호 서식에 따른 전문(全文) 보정된 명세서(청구범위 포함)·요약서 및 필요한 도면을 첨부하여 제출해야 합니다.

1/2

향후계획.
+Q&A